

Lab 12 - Governance and incident recovery

AI Agents Fundamentals for Beginners | C690 | v1.0

Objective

Test denied actions, approval binding and a stop/recovery plan.

Alignment: LO4. Scheduled hands-on time: 35 minutes, with trainer-led interpretation alongside the slides.

Files and prerequisites

- `attack-cases.csv`
- `access-policy.json`
- `incident-template.json`

Use Python 3.10 or newer and an organisation-approved LLM chat interface. The core local run requires no API key. Model outputs vary: assess evidence and behaviour, not matching prose. All business records are synthetic.

Detailed procedure

1. Open `access-policy.json` and `attack-cases.csv`. All test attacks are synthetic strings for local validation; they are not instructions to access real systems.
2. Run `python3 run.py`. Inspect the actual deny results for cross-customer reads, unapproved writes and disabled execution.
3. Use prompt 1 to analyse each case and propose a response within the policy. Compare the model response with the enforced code decision.
4. Use prompt 2 to rehearse a timeout after a simulated write. Require transaction readback before any retry and explain the idempotency key.
5. Complete `incident-template.json` with containment, evidence, owner, rollback, in-flight work and regression test. Include a named role rather than an invented real person.
6. Use prompt 3 to review readiness for a 5-percent draft-only canary. Save the completed incident record and state conditions that prohibit scale-up.

Acceptance checks

- Unapproved writes and cross-customer reads are denied by code.
- Stop state prevents execution regardless of model output.
- Recovery checks transaction state before retry and assigns an owner.

Run `python verify.py` after `run.py` (use `python3` if that is your interpreter command). The script verifies deterministic mechanics. Separately complete the evidence-based review of your own LLM output; no script claims an LLM task passed unless its output was actually inspected.

Troubleshooting

- Missing package: confirm the virtual environment is active, then install this folder's `requirements.txt`.
- File not found: open a terminal in this exact lab folder; the script resolves input paths relative to itself.
- Invalid JSON: remove Markdown fences and save a single JSON value; keep the original response for diagnosis.

- Unreliable model answer: reduce the task, include source IDs, inspect each claim and escalate missing evidence.
- Training results differ: record Python/package versions and seed; compare trends and limitations rather than claiming identical scores.

Evidence and cleanup

Keep output.json, learner-output.json and relevant screenshots or logs in your learner submission folder. Stop after verification; do not connect the exercise to real payment, email, HR or production systems. Local generated outputs can be archived after the lab; keep source data and prompts unchanged.